Comparative Study of Parallelization Techniques for Word Counting Map Reduce Algorithm

Kuan Hsi Chu
National Yang Ming Chiao Tung University
Taiwan

Tzu Jung Chen
National Yang Ming Chiao Tung University
Taiwan

Hsiao Hsin Lin
National Yang Ming Chiao Tung University
Taiwan

ABSTRACT

This project seeks to delve into the diverse implementation strategies, performance dynamics, and development hurdles associated with various parallelization techniques applied to a common computational task: Word Counting. Our approach involves the creation of four distinct custom frameworks that emulate the MapReduce[1] paradigm, utilizing Pthreads, OpenMP, MPI, and CUDA. Each methodology embodies a unique model of parallelization: Pthreads and OpenMP facilitate multi-threaded parallelism on CPUs, MPI enables distributed computation across multiple nodes, and CUDA leverages the capabilities of GPUs for data-parallel operations. Through this comparative study, we will thoroughly examine how each framework approaches the essential Map, Shuffle, and Reduce phases of the computation process. Our analysis aims to illuminate the inherent trade-offs related to communication overhead, memory management, and scalability, providing valuable insights into the effectiveness and efficiency of each parallelization technique in the context of our core application.

1 INTRODUCTION

High-performance computing (HPC) and big data processing frequently rely on the MapReduce paradigm for its robust and scalable nature. While industrial frameworks like Hadoop and Spark abstract the parallelization details, a deep understanding of the underlying hardware and programming models is crucial for performance tuning. This project aims to strip away that abstraction by implementing the core MapReduce workflow (Map, Shuffle, Reduce) using four fundamental parallel programming models: MPI (Distributed Memory), OpenMP (Shared Memory CPU), Pthreads (Shared Memory CPU), and CUDA (Data-Parallel GPU)[2]. By comparing these four distinct implementations on the identical computational problem, we can quantitatively assess the intrinsic performance trade-offs, scalability limits, and architectural bottlenecks of each paradigm when simulating a real-world big data workload.

2 PROPOSED SOLUTION

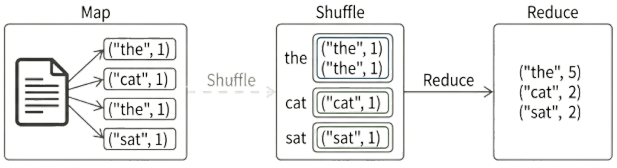


Figure 1: Architecture of MapReduce

Integration and Interaction: As shown in Figure 1, a master component coordinates data segmentation, assignment, and result aggregation across all approaches, while worker components handle the core computation. The interactions involve distributing data, computing local results, and merging outputs into a global histogram. This architecture ensures consistency in results across all implementations and facilitates a direct performance comparison.

The project will implement four different parallelized MapReduce-style word count solutions.

2.1 Pthreads and OpenMP

The text dataset will be divided into multiple segments. Each thread will process one segment, generating a sequence of intermediate key-value pairs. These pairs will then be grouped and aggregated into a global histogram during the Reduce phase. OpenMP facilitates shared-memory parallelism, simplifying the implementation, while Pthreads allows finer control over thread creation and synchronization.

2.2 MPI

The dataset will be distributed across multiple processes in a cluster environment. Each process counts words in its assigned portion of the dataset. MPI collective communication routines, such as MPI_Reduce or MPI_Gather, will aggregate results into a global histogram. This approach highlights distributed computation challenges and communication overhead.

2.3 CUDA

The dataset will be divided into blocks assigned to GPU threads. Each thread will perform word counting on its block, storing intermediate results in shared memory. A parallel reduction algorithm will merge local histograms into a global result. Special attention will be given to handle word boundaries that span multiple blocks.

3 HOW PARALLELIZATION IS DONE

3.1 Overview

We implement four distinct parallel approaches to the MapReduce Word Count problem: Pthreads, OpenMP, MPI, and CUDA. While all implementations adhere to the canonical MapReduce workflow—comprising Map, Shuffle, and Reduce phases—they employ divergent parallelization strategies optimized for their respective hardware characteristics. The system architectures for the CPU-based methods (Pthreads, OpenMP, and MPI) are illustrated in Figure 2, highlighting their shared and distributed memory mechanisms. Conversely, the architecture for the GPU-based approach (CUDA) is depicted in Figure 3.

3.2 Pthread Implementation

The Pthread implementation adopts a persistent thread pool model to avoid thread creation overhead across phases. Threads follow an SPMD execution model and synchronize using explicit barrier mechanisms between phases. In the map phase, the input text is partitioned into contiguous chunks with boundary adjustment to prevent word splitting. Each thread processes its chunk independently and stores results in thread-local buffers, eliminating contention. During the shuffle phase, threads hash-partition their local outputs into thread-specific partitions without synchronization. In the reduce phase, each thread aggregates data assigned to its partition using a local hash map, ensuring lock-free reduction. This design minimizes synchronization overhead and maximizes cache locality.

3.3 OpenMP Implementation

The OpenMP implementation relies on compiler directives to manage parallelism, favoring simplicity and maintainability. Threads operate in a shared-memory model with implicit synchronization. The map phase uses static scheduling to evenly divide input data across threads with minimal overhead. The shuffle phase performs hash-based partitioning using thread-private output regions. The reduce phase employs dynamic scheduling to mitigate load imbalance caused by uneven key distribution. OpenMP's implicit barriers and runtime scheduling reduce code complexity while maintaining competitive performance.

3.4 MPI Implementation

The MPI implementation targets distributed-memory execution using explicit message passing. Although experiments are conducted on a single node, the design is extensible to multi-node clusters. Rank 0 serves as the coordinator to determine the total file size and partitioning logic. Instead of using MPI_Scatterv() to distribute the raw data, Rank 0 calculates the file offsets and sizes for each process and broadcasts this metadata. Each process then independently utilizes seekg() to perform Parallel I/O, reading its assigned portion of the dataset concurrently from the shared storage. The shuffle phase uses MPI_Alltoallv() to redistribute key-value pairs among processes based on hash partitioning. Each process then performs the reduce phase locally, and final results are gathered using MPI_Gatherv(). MPI collective operations provide scalable synchronization and communication at the cost of increased implementation complexity.

3.5 CUDA Implementation

The CUDA implementation exploits massive GPU parallelism to process large inputs. The map phase uses a two-stage approach: parallel word boundary detection followed by word processing, improving thread utilization and reducing divergence. The shuffle phase constructs key-grouped data using a global hash table with atomic operations. The reduce phase assigns one thread per key group to aggregate counts independently. Design choices such as grid-stride loops, constant memory for stopwords, and careful use of atomic operations balance performance and correctness on the GPU.

3.6 Comparison of Parallelization Strategies

The four implementations represent distinct parallelization philosophies. Pthread and OpenMP leverage shared-memory parallelism with low communication overhead; MPI enables scalability across distributed systems through message passing; CUDA achieves high throughput through massive fine-grained parallelism on GPUs. Each approach presents trade-offs among performance, scalability, and implementation complexity, making the choice of strategy dependent on available hardware resources and application requirements.

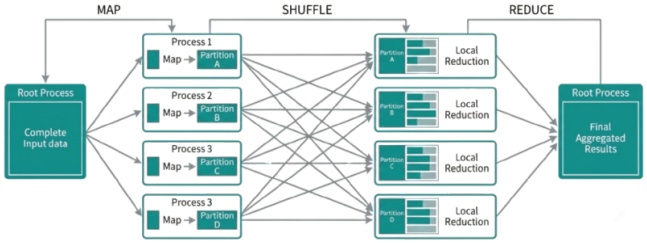


Figure 2: CPU base flow chart

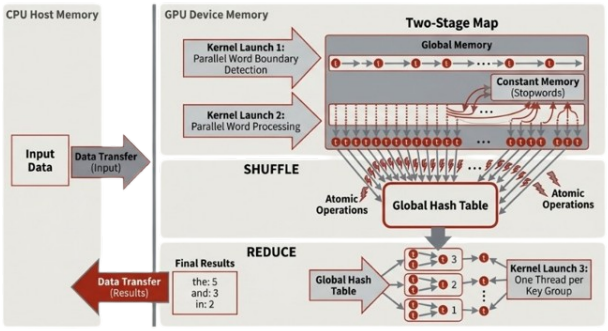


Figure 3: GPU base flow chart

4 EXPERIMENTAL METHODOLOGY

4.1 Datasets

To evaluate the scalability and performance of different parallelization strategies, we employed three datasets of varying magnitudes:

- **Small:** `pg11.txt` (170 KB)[3]
- **Medium:** `text8.txt` (100 MB)[4]
- **Large:** `enwik9.txt` (1 GB)[5]

4.2 Experimental Design

We conducted a comprehensive analysis to discuss the impact of various parallelization methods across these data scales. To ensure statistical reliability and minimize the impact of system jitter, each experiment was executed 10 times for every combination of parallelization method and dataset. The reported results represent the arithmetic mean of these execution times.

4.3 System Specifications

All experiments were conducted in the following hardware and software environment:

Operating System: Windows Subsystem for Linux 2 (WSL 2) running Ubuntu 24.04.3 LTS
Kernel: 6.6.87.2-microsoft-standard-WSL2
CPU: 13th Gen Intel® Core™ i5-13500HX (14 Cores, 20 Threads)
GPU: NVIDIA GeForce RTX 4060 (CUDA 13.0)

5 EXPERIMENTAL RESULTS

5.1 Total Speedup Analysis

To evaluate the effectiveness of the proposed parallel MapReduce frameworks, we measured the total execution speedup relative to the sequential baseline ($Speedup = T_{seq}/T_{parallel}$) across three datasets of varying sizes: pg11.txt (Small), text8 (Medium), and enwik9 (Large). The results are illustrated in Figure 4.

5.1.1 Overall Speedup and Scalability. As observed in the left panel of Figure 4, parallelization yields negligible benefits for the small dataset. In many configurations, the speedup drops below 1.0, indicating that the parallel version is slower than the sequential one. This phenomenon is attributed to the high ratio of parallel management overhead (thread creation, context switching, and synchronization barriers) relative to the actual computational workload. For small inputs, the execution time is dominated by the initialization costs rather than the text processing tasks, validating Amdahl's Law where the non-parallelizable portion of the code limits the theoretical maximum speedup. For the medium and large datasets (center and right panels of Figure 4), the computational workload increases sufficiently to amortize the parallelization overhead. A clear trend of improved scalability is visible. As the number of processors (N_P) increases, the speedup for Pthreads and OpenMP grows significantly, approaching the ideal linear speedup in several instances. This confirms that as the data volume grows, the system shifts from an overhead-dominated regime to a computation-dominated regime, where parallel processing becomes highly effective.

5.1.2 Comparative Analysis of Architectures. A consistent finding across our experiments is that the shared-memory CPU implementations (Pthreads and OpenMP) generally offer the most robust performance for this specific Word Count workload. As shown in the enwik9 results, both Pthreads and OpenMP achieve the highest peak speedups (exceeding 30x on high thread counts). These architectures benefit from low-latency access to shared memory, minimizing the need for expensive data copying or inter-process communication (IPC) which hampers the MPI implementation.

The CUDA implementation (represented by the red dashed line) exhibits distinct behavior:

Scalability Limitations: While CUDA demonstrates strong performance on the medium dataset (text8), it is outperformed by CPU multi-threading on the largest dataset (enwik9) when CPU core counts are high.

Bottleneck Analysis: Although GPUs offer massive parallelism for the Map phase, the performance is likely constrained by PCIe bandwidth bottlenecks during memory transfer (Host-to-Device and Device-to-Host) and the complexity of the Reduce phase on GPU architectures (handling atomic operations and memory divergence). Consequently, while CUDA shows potential scalability for massive data, the overhead of memory transfer and kernel launch latencies prevents it from surpassing the optimized Pthreads/OpenMP versions in the total execution time metric.

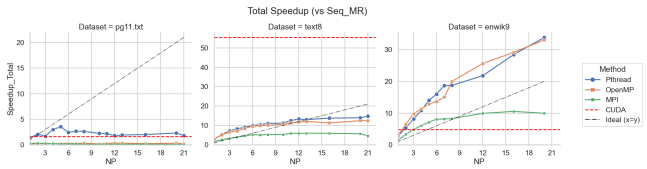


Figure 4: Total speedup comparison

5.2 Map Phase Performance Analysis

Figure 5 presents the speedup specifically for the Map phase across the three datasets. Unlike the Reduce phase, the Map phase is characterized by compute-intensive string parsing and tokenization. While the small dataset (pg11.txt) exhibits the expected lack of speedup due to initialization dominance, the medium and large datasets reveal distinct architectural behaviors and hardware saturation points.

5.2.1 GPU Scalability and Memory Bottlenecks. A significant performance divergence is observed in the CUDA implementation (red dashed line) between the medium and large datasets.

- **Peak Efficiency on Medium Data:** On the text8 dataset, CUDA achieves a remarkable speedup of approximately 60×, drastically outperforming CPU methods. This indicates that for mid-sized data (~100MB), the GPU's massive parallelism successfully hides memory access latency, and the computational intensity of tokenization is sufficient to saturate the device's compute units without hitting memory ceilings.
- **Degradation on Large Data:** Conversely, on the 1GB enwik9 dataset, the CUDA speedup collapses to approximately 4×. We attribute this drastic degradation to *Global Memory Bandwidth Saturation* and *PCIe Bottlenecks*. As the dataset size exceeds the GPU's on-chip cache capacity, the kernel suffers from frequent global memory transactions. Furthermore, the character-level branching required for delimiter checks causes *warp divergence*, while the sheer volume of data transfer over PCIe exposes I/O bottlenecks that negate the computational advantage of the GPU cores during the Map phase.

5.2.2 CPU Saturation and Heterogeneous Architecture Effects. For the shared-memory CPU implementations (Pthreads and OpenMP), we observe a "saturation point" in the medium dataset where speedup plateaus early, limiting the scalability compared to the ideal linear growth.

- **The Straggler Effect (P-Cores vs. E-Cores):** The plateau observed around $N_P = 6$ in the medium dataset correlates strongly with the physical core topology of our test environment, which consists of 6 Performance Cores (P-Cores).

Since our implementation employs *static data partitioning* (dividing the file size equally by N_P), the workload does not account for processor heterogeneity. When $N_P > 6$, the scheduler utilizes Efficient Cores (E-Cores). The faster P-Cores complete their assigned chunks earlier and must idle while waiting for the slower E-Cores to finish, creating a *straggler effect* that significantly dampens the speedup curve.

- **Memory Bandwidth Saturation:** Additionally, the Map phase involves intensive linear memory reads. In the medium dataset, the multiple threads likely saturate the available DRAM bandwidth quickly, leading to a memory-bound state. In the larger enwik9 dataset, the sheer volume of parsing logic allows the CPU to maintain a slightly higher computation-to-memory-access ratio, resulting in the smoother, more linear scaling curve observed in the rightmost panel of Figure 5.

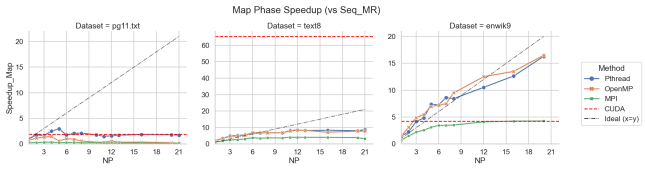


Figure 5: Map speedup comparison

5.3 Shuffle Phase Performance Analysis

As shown in Figure 6, the dataset size significantly influences the speedup efficiency across different parallel architectures during the Shuffle phase:

- **Small Dataset (pg11.txt)**: The speedup for all models fluctuates significantly.For very small workloads, the computational gain is outweighed by the parallelization overhead, such as thread management and synchronization.
- **Medium Dataset (text8)**: **CUDA** exhibits superior performance compared to CPU-based models. This efficiency stems from our custom **Atomic Hash Table** implementation, which bypasses time-consuming sorting operations ($O(N)$ vs. $O(N \log N)$) during data grouping.
- **Large Dataset (enwik9)**:
  - **GPU Limitations**: **CUDA**'s speedup performance declines as the data size exceeds hardware thresholds. The massive Hash Table required for enwik9 likely exceeds the **GPU L2 Cache** capacity, leading to increased memory latency.
  - **Shared Memory Efficiency (Pthreads/OpenMP)**: Both **Pthreads** and **OpenMP** demonstrate stable linear scalability.By utilizing **Thread-Local Storage (TLS)**, each thread operates within its local cache without lock contention, making shared memory architectures highly effective for large-scale Shuffle operations.
  - **Distributed Memory (MPI)**: Although **MPI** shows steady growth, its speedup remains lower than shared memory models.The bottleneck persists due to the high **inter-process communication (IPC) overhead** required to aggregate distributed results, even as the computational load increases.

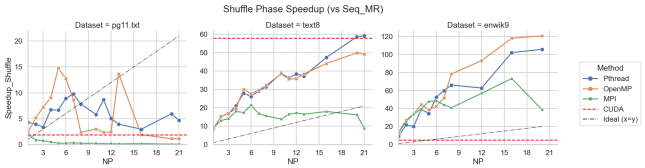


Figure 6: Shuffle speedup comparison

5.4 Reduce Phase Analysis

Figure 7 and Figure 8 illustrate the performance characteristics of the Reduce phase across different dataset sizes and parallelization approaches. The Reduce phase involves aggregating word counts through hash table lookups and summation operations, revealing a distinct performance gap between GPU and CPU architectures.

5.4.1 The Dominance of GPU Acceleration. As shown in Figure 7, the CUDA implementation demonstrates overwhelming superiority for larger datasets, achieving a speedup of approximately 6.5× on text8 and peaking at nearly 38× on enwik9. This massive performance gain validates the design strategy of assigning "one thread per hash table slot" for independent aggregation. The Reduce operation is embarrassingly parallel and memory-intensive but computationally simple—each thread sums values in its assigned key group without inter-thread communication. The GPU effectively hides memory latency through massive thread-level parallelism, processing thousands of key groups simultaneously without the synchronization overhead that plagues CPU threads. However, CUDA's advantage is absent for small datasets: on pg11.txt, CUDA achieves only 0.5× speedup, performing worse than the sequential baseline. This confirms that for small workloads, the overhead of memory transfer and kernel launch outweighs the benefits of parallelism.

5.4.2 CPU Scalability Challenges. In contrast to CUDA, the CPU-based approaches (Pthread, OpenMP, MPI) struggle to achieve meaningful speedup, appearing as near-flat lines in Figure 7. To investigate this behavior in detail, Figure 8 provides a zoomed-in view excluding CUDA results. Sublinear Speedup on Medium Data: Figure 8(a) reveals that for the text8 dataset, CPU parallelization provides minimal benefits. Pthread achieves the best performance at approximately 0.8X speedup with NP=21, remaining below the sequential baseline. OpenMP reaches only 0.55X, while MPI plateaus near 0.2×. The limited speedup indicates that parallelization overhead—including thread management, barrier synchronization, and memory contention—exceeds the time saved by the lightweight summation task. Modest Gains on Large Data: Positive scalability emerges only on the largest dataset at higher thread counts, as shown in Figure 8(b). Pthread surpasses the sequential baseline (Speedup > 1.0) beginning around NP=12, peaking at approximately 1.4X with NP=20. This represents only 7% parallel efficiency, indicating persistent bottlenecks. OpenMP achieves approximately

1.05X speedup at NP=18-20, with dynamic scheduling partially mitigating load imbalance but introducing scheduling overhead. MPI consistently performs worst, peaking at only 0.35× due to the serialization bottleneck of MPI_Gatherv, where all reduced results must be transmitted to a single rank.

5.4.3 Summary of Reduce Phase Performance. In summary, the experimental results indicate that CPU parallelization for the Reduce phase is only marginally viable for extremely large datasets (≥ 1 GB), where the workload can partially amortize synchronization costs. In contrast, GPU acceleration proves to be highly effective for medium to large datasets, delivering a 6-38X speedup. The data suggests that for MapReduce workloads with skewed key distributions, GPU acceleration is the superior architectural choice for the Reduce phase.

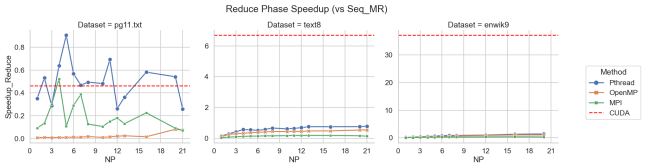
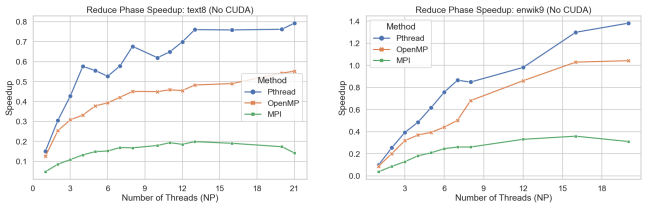


Figure 7: Reduce speedup comparison



(a) text8 Reduce Speedup **(b) enwik9 Reduce Speedup**

Figure 8: Reduce Phase Speedup without CUDA

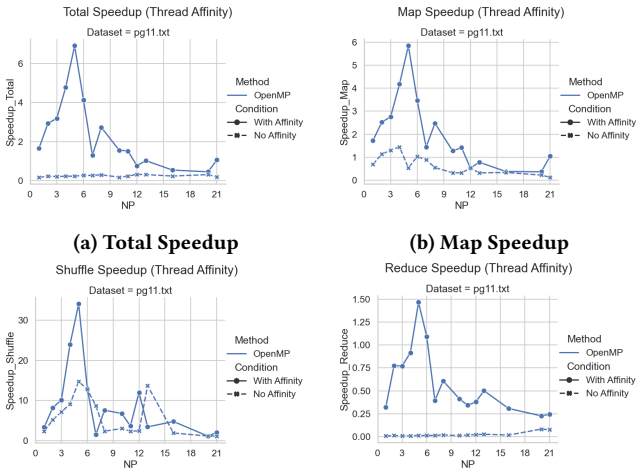
5.5 Summary of Analysis

In conclusion, while parallelization is ineffective for small datasets due to inherent overhead, it provides substantial gains for large-scale text processing. Despite the theoretical throughput advantages of GPUs, our experimental results indicate that Pthreads and OpenMP are the superior choices for this MapReduce Word Count implementation, offering the best balance of low synchronization cost and high scalability on the largest tested datasets.

6 ABLATION STUDY

In this study, we evaluate the performance impact of Thread Affinity (Pinning) within the OpenMP framework, specifically using a small-scale dataset (pg11.txt). As shown in Figure 9, conducting experiments on a small dataset highlights how explicitly binding threads to specific processor cores (With Affinity) significantly outperforms default operating system scheduling (No Affinity) by minimizing frequent context switching and thread migration overhead.

- **Total Speedup (Fig. 9a)**: The "With Affinity" mode consistently outperformed the default scheduling across all stages.
- **Map Phase Efficiency (Fig. 9b)**: By pinning threads to specific processes, the Map phase achieved a peak speedup of nearly 6x. In contrast, the version without affinity stayed near 1x, as OS-level context switching overhead neutralized parallel gains.
- **Shuffle Phase Breakthrough (Fig. 9c)**: The most significant improvement was observed in the Shuffle phase, where thread pinning reached a speedup of approximately 30x. This demonstrates that maintaining thread-to-core consistency significantly enhances cache locality during intensive data reorganization.
- **Reduce Phase Gain (Fig. 7d)**: The Reduce phase also showed a distinct performance advantage when thread affinity was enabled, peaking at a 1.5x speedup, whereas the unpinned version showed minimal efficiency.



(a) Total Speedup **(b) Map Speedup**

(c) Shuffle Speedup **(d) Reduce Speedup**

Figure 9: OpenMP Thread Affinity

7 RELATED WORK

Word Count is commonly used as a benchmark in parallel computing research. MPI-based approaches demonstrate distributed computation efficiency, while Pthreads and OpenMP implementations provide insight into multi-threaded CPU performance. CUDA implementations illustrate massive parallelism on GPUs. High-level frameworks like Hadoop or Spark offer MapReduce [1] abstractions. Yadav el. [6] implement Word-Count with Map Reduce Programming Using Hadoop Framework. This project builds upon these studies by creating all four MapReduce implementations from scratch, also we do the performance comparisons like [7] [8] and revealing the underlying design and parallelization challenges.

8 CONTRIBUTIONS OF EACH MEMBER

- Kuan Hsi Chu : Presentation Preparation, MPI Implementation, Final Report Compilation

- Tzu Jung Chen : Presentation Preparation, OpenMP and Pthread Implementation, Final Report Compilation
- Hsiao Hsin Lin : Presentation Preparation, CUDA Implementation, Final Report Compilation

9 CONCLUSION

This study evaluated the performance of four distinct parallel programming models—Pthreads, OpenMP, MPI, and CUDA—implemented within a MapReduce framework for Word Counting. Based on the experimental results across varying dataset sizes (pg11.txt, text8, and enwik9), several key insights were observed:

- **Performance vs. Dataset Size**: Parallelization efficiency is highly dependent on workload volume. For small datasets like pg11.txt, the overhead outweighs computational benefits.
- **CPU Optimization**: We eliminated lock contention on global data structures by implementing Thread-Local Storage (TLS).
- **Distributed Scalability**: Overcame the single-rank I/O bottleneck by implementing Parallel I/O in MPI.
- **GPU Acceleration**: Replaced sorting-based reduction with a custom Atomic Hash Table, significantly improving throughput for CUDA.

REFERENCES

[1] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Commun. ACM*, 51(1):107–113, January 2008.

[2] Peter Pacheco and Matthew Malensek. *An introduction to parallel programming*. Morgan Kaufmann, 2021.

[3] Project Gutenberg. Small dataset (170 kb): pg11.txt. https://www.gutenberg.org/ebooks/11. Accessed: 2025-12-19.

[4] York Yong. Medium dataset (100 mb): text8.zip. https://www.kaggle.com/datasets/yorkyong/text8-zip, 2024.

[5] Matt Mahoney. Large dataset (1 gb): enwik9. https://mattmahoney.net/dc/textdata.html.

[6] Santosh Yadav and Jay Prakash. A survey on implementation of word-count with map reduce programming oriented model using hadoop framework. *SSRN Electronic Journal*, 01 2019.

[7] Zahra Mohamed and Abdulla AlQaddoumi. Comparing sequential and parallel word counting methods in finding commonly used words by different authors. pages 1–4, 2020.

[8] Sol Ji Kang, Sang Yeon Lee, and Keon Myung Lee. Performance comparison of openmp, mpi, and mapreduce in practical problems. *Advances in Multimedia*, 2015(1):575687, 2015.